

Assignment 1 – PS4: Defence Drone Navigation

BITS Pilani WILPD | MTech in AI & ML | S2 2025-2026

Course: AIMLCZG557 / AECLZG557 | Group ID: G244

Problem Summary

An autonomous UAV (drone) must navigate an **8×8 grid** from **Start S=(0,0)** to **Goal E=(6,7)**

using **Greedy Best-First Search (GBFS)** with two heuristics, avoiding No-Fly Zones (N).

```
In [1]: # — Cell 1 : Imports & PEAS Description —————
import heapq, math, time, tracemalloc, os
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

peas = {
    'Performance': [
        'Reach Goal node E at (6,7) from Start S at (0,0)',
        'Minimise heuristic cost; avoid No-Fly Zones',
        'Scoring: S=+2, E=+2, .=+1, W=+4, N=+8 (blocked)',
    ],
    'Environment': [
        '8x8 discrete 2-D grid airspace',
        'Cell types: S (start), E (goal), . (passable), W (weather hazard), N (no-fly zone)',
        'Fully observable, deterministic, static, discrete',
    ],
    'Actuators': [
        'Move North (row-1)', 'Move South (row+1)',
        'Move East (col+1)', 'Move West (col-1)',
    ],
    'Sensors': [
        'Current grid position (row, col)',
        'Cell type of all adjacent nodes',
        'Heuristic value at each neighbour',
    ],
}

print('=' * 62)
print(' PEAS Description - Defence Drone Autonomous Agent (G244) ')
print('=' * 62)
for key, items in peas.items():
    print(f'\n {key}:')
    for item in items:
        print(f'     • {item}')
print('=' * 62)
```

=====

PEAS Description - Defence Drone Autonomous Agent (G244)

=====

Performance:

- Reach Goal node E at (6,7) from Start S at (0,0)
- Minimise heuristic cost; avoid No-Fly Zones
- Scoring: S=+2, E=+2, .=+1, W=+4, N=+8 (blocked)

Environment:

- 8x8 discrete 2-D grid airspace
- Cell types: S (start), E (goal), . (passable), W (weather hazard), N (no-fly)
- Fully observable, deterministic, static, discrete

Actuators:

- Move North (row-1)
- Move South (row+1)
- Move East (col+1)
- Move West (col-1)

Sensors:

- Current grid position (row, col)
 - Cell type of all adjacent nodes
 - Heuristic value at each neighbour
- =====

```
In [2]: # — Cell 2 : Grid Setup & Initial Visualisation —————
INPUT_FILE = os.path.join(os.path.dirname(os.path.abspath('__file__')), 'inputP
OUTPUT_FILE = os.path.join(os.path.dirname(os.path.abspath('__file__')), 'output

def load_grid(path):
    with open(path) as f:
        lines = [l.strip() for l in f if l.strip()]
        rows, cols = map(int, lines[0].split())
        grid = [lines[i + 1].split() for i in range(rows)]
        sr, sc = map(int, lines[rows + 1].split())
        gr, gc = map(int, lines[rows + 2].split())
        return grid, rows, cols, (sr, sc), (gr, gc)

RAW_GRID, ROWS, COLS, START, GOAL = load_grid(INPUT_FILE)

CELL_COST = {'S': 2, 'E': 2, '.': 1, 'W': 4, 'N': 8}
H2_CELL_COST = {'S': 1, 'E': 1, '.': 1, 'W': 4, 'N': 8} # per assignment exampl

# North > East > South > West (tie-break priority)
DIRECTIONS = [(-1, 0, 'N'), (0, 1, 'E'), (1, 0, 'S'), (0, -1, 'W')]
DIR_PRI = {'N': 0, 'E': 1, 'S': 2, 'W': 3}

CELL_COLOR = {'S': '#27ae60', 'E': '#c0392b', '.': '#ecf0f1',
               'W': '#e67e22', 'N': '#2c3e50'}

def visualize_grid(grid, path=None, explored=None, frontier=None, title='Grid',
fig, ax = plt.subplots(figsize=(9, 9))
path_set = set(path) if path else set()
explored_set = set(explored) if explored else set()
frontier_set = set(frontier) if frontier else set()
for r in range(ROWS):
    for c in range(COLS):
        cell = grid[r][c]
```

```

        color = CELL_COLOR[cell]
        if (r, c) in path_set and cell not in ('S', 'E'): color = '#29
        elif (r, c) in explored_set and cell not in ('S', 'E'): color = '#bd
        elif (r, c) in frontier_set and cell not in ('S', 'E'): color = '#f1
        rect = mpatches.FancyBboxPatch((c, ROWS - 1 - r), 1, 1,
            boxstyle='round,pad=0.04', facecolor=color, edgecolor='#7f8c8d',
        ax.add_patch(rect)
        label = cell
        if h_vals and (r, c) in h_vals:
            label = f'{cell}\n{h_vals[(r,c)]:.1f}'
        elif path and (r, c) in path_set:
            idx = list(path).index((r, c))
            label = f'{cell}\n{idx}' if cell in ('S', 'E') else str(idx)
        ax.text(c + 0.5, ROWS - 1 - r + 0.5, label, ha='center', va='center',
            fontsize=9, fontweight='bold',
            color='white' if cell == 'N' else '#1a1a2e')
    ax.set_xlim(0, COLS); ax.set_ylim(0, ROWS)
    ax.set_xticks(range(COLS)); ax.set_yticks(range(ROWS))
    ax.set_xticklabels(range(COLS))
    ax.set_yticklabels(range(ROWS - 1, -1, -1))
    ax.set_xlabel('Column', fontsize=12); ax.set_ylabel('Row', fontsize=12)
    ax.set_title(title, fontsize=13, fontweight='bold', color='#2c3e50')
    legend_elements = [
        mpatches.Patch(facecolor='#27ae60', label='S - Start (0,0)'),
        mpatches.Patch(facecolor='#c0392b', label='E - Goal (6,7)'),
        mpatches.Patch(facecolor='#ecf0f1', edgecolor='grey', label='. - Passabl
        mpatches.Patch(facecolor='#e67e22', label='W - Weather Hazard (cost 4)'),
        mpatches.Patch(facecolor='#2c3e50', label='N - No-Fly Zone (blocked)'),
        mpatches.Patch(facecolor='#2980b9', label='Path'),
        mpatches.Patch(facecolor='#bdc3c7', label='Explored'),
        mpatches.Patch(facecolor='#f1c40f', label='Frontier'),
    ]
    ax.legend(handles=legend_elements, loc='upper right',
        bbox_to_anchor=(1.35, 1.01), fontsize=9)
    plt.tight_layout()
    plt.show()

print(f'Grid loaded: {ROWS}x{COLS} | Start={START} | Goal={GOAL}')
print('Grid:')
for i, row in enumerate(RAW_GRID):
    print(f' Row {i}: {row}')
visualize_grid(RAW_GRID, title='Initial 8x8 Grid - Defence Drone Environment (G2

```

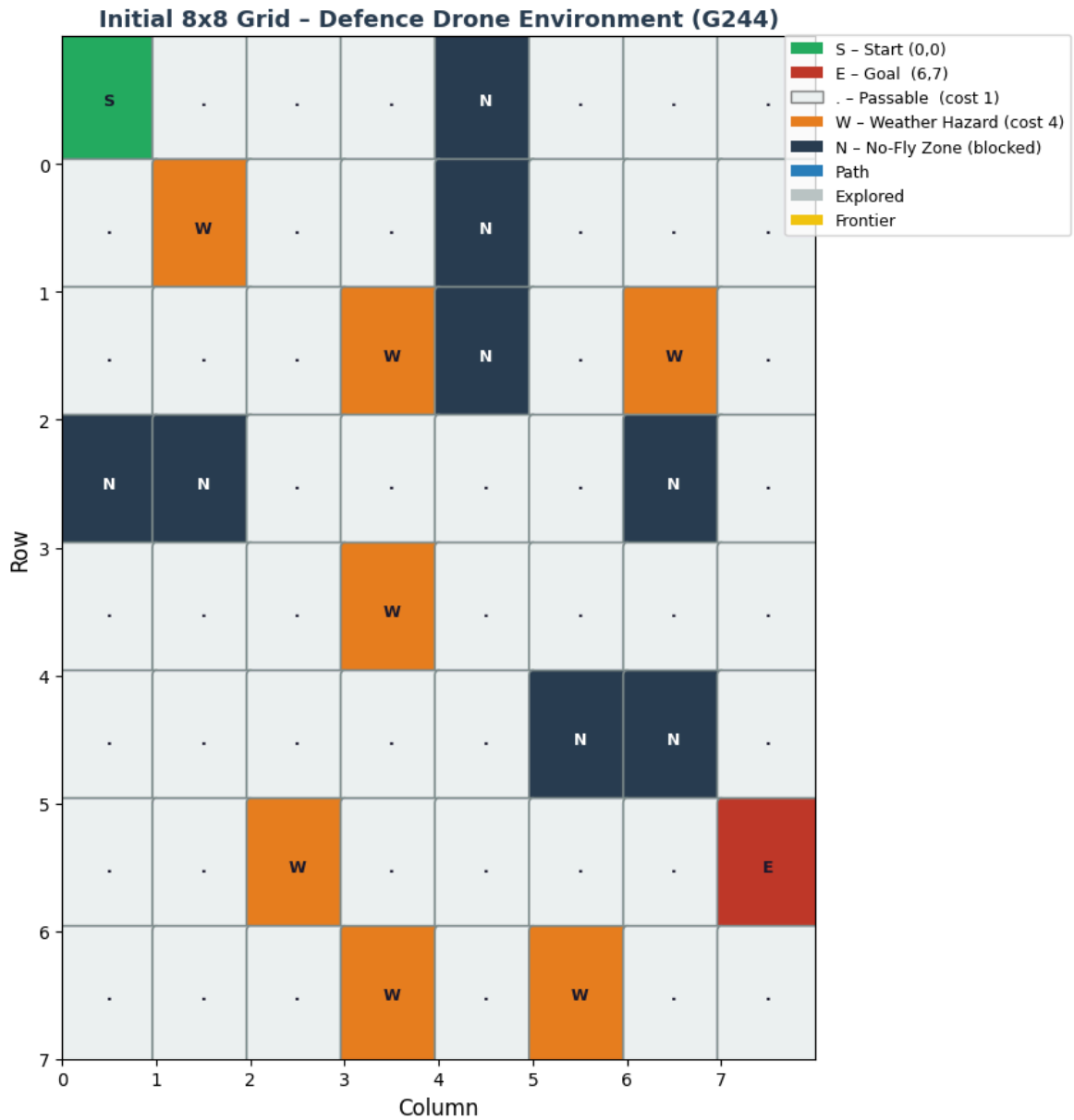
Grid loaded: 8x8 | Start=(0, 0) | Goal=(6, 7)

Grid:

```

Row 0: ['S', '.', '.', '.', 'N', '.', '.', '.']
Row 1: ['.', 'W', '.', '.', 'N', '.', '.', '.']
Row 2: ['.', '.', '.', 'W', 'N', '.', 'W', '.']
Row 3: ['N', 'N', '.', '.', '.', '.', 'N', '.']
Row 4: ['.', '.', '.', 'W', '.', '.', '.', '.']
Row 5: ['.', '.', '.', '.', '.', 'N', 'N', '.']
Row 6: ['.', '.', 'W', '.', '.', '.', '.', 'E']
Row 7: ['.', '.', '.', 'W', '.', 'W', '.', '.']

```



```
In [3]: # — Cell 3 : Heuristic Functions —————

def h1(node, goal=None):
    """h1: Euclidean distance to goal."""
    g = goal if goal else GOAL
    return math.sqrt((node[0] - g[0]) ** 2 + (node[1] - g[1]) ** 2)

def h2(node, goal=None, grid=None):
    """h2: Bounding-Box Risk-Weighted Heuristic (future-aware).
    h2(n) = Manhattan_dist * (1/K * sum of cell costs in bounding box)
    """
    g = goal if goal else GOAL
    gr = grid if grid else RAW_GRID
    manhattan = abs(node[0] - g[0]) + abs(node[1] - g[1])
    if manhattan == 0:
        return 0.0
    r_min, r_max = min(node[0], g[0]), max(node[0], g[0])
    c_min, c_max = min(node[1], g[1]), max(node[1], g[1])
    K = (r_max - r_min + 1) * (c_max - c_min + 1)
    total = sum(H2_CELL_COST[gr[r][c]]
                for r in range(r_min, r_max + 1)
                for c in range(c_min, c_max + 1))
    return manhattan * (total / K)
```

```

# — Verify with assignment example: node (3,3) —
node_ex = (3, 3)
manh     = abs(3 - 6) + abs(3 - 7)
K_ex     = (6 - 3 + 1) * (7 - 3 + 1)
box_sum  = sum(H2_CELL_COST[RAW_GRID[r][c]] for r in range(3, 7) for c in range(3, 7))
h2_ex    = h2(node_ex)
print('=== Heuristic Verification (Assignment Example: node (3,3)) ===')
print(f' Manhattan |3-6|+|3-7| = {manh}')
print(f' Box rows 3-6, cols 3-7 K={K_ex} cells, sum={box_sum}')
print(f' Avg window cost = {box_sum}/{K_ex} = {box_sum/K_ex:.2f}')
print(f' h2(3,3) = {manh} x {box_sum/K_ex:.2f} = {h2_ex:.2f} (expected 15.40)')
print(f' h1(3,3) = sqrt({manh**2-1}+16) = {h1(node_ex):.4f}')
print()

# — Heatmaps —
h1_map = np.array([[h1((r, c)) for c in range(COLS)] for r in range(ROWS)])
h2_map = np.array([[h2((r, c)) for c in range(COLS)] for r in range(ROWS)])

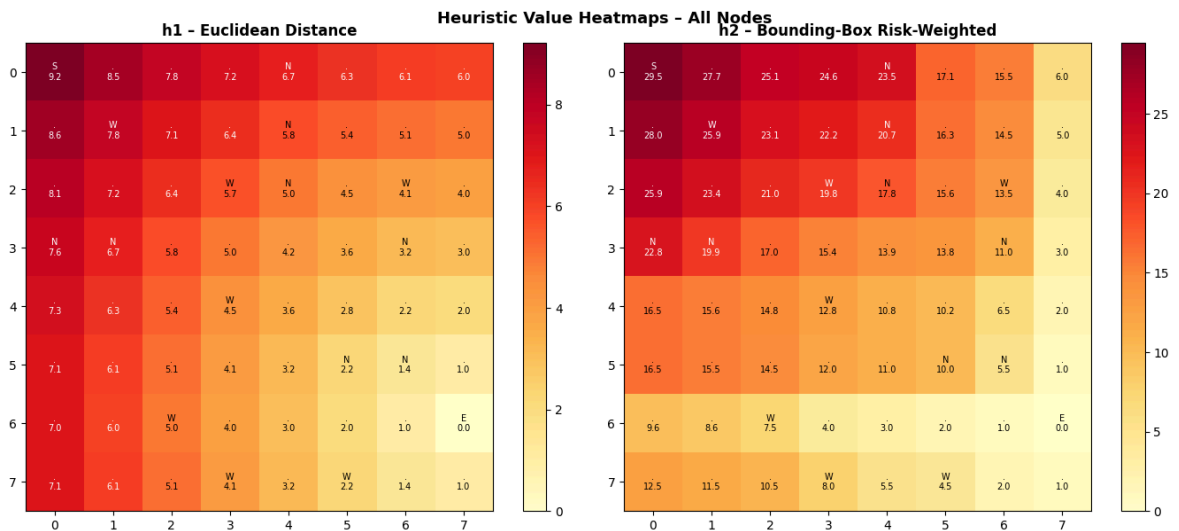
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, vals, title in [(axes[0], h1_map, 'h1 - Euclidean Distance'),
                        (axes[1], h2_map, 'h2 - Bounding-Box Risk-Weighted')]:
    im = ax.imshow(vals, cmap='YlOrRd', origin='upper')
    for r in range(ROWS):
        for c in range(COLS):
            cell = RAW_GRID[r][c]
            txt = f'{vals[r, c]:.1f}'
            color = 'white' if vals[r, c] > vals.max() * 0.65 else 'black'
            ax.text(c, r, f'{cell}\n{txt}', ha='center', va='center',
                    fontsize=7, color=color)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xticks(range(COLS)); ax.set_yticks(range(ROWS))
    plt.colorbar(im, ax=ax)
plt.suptitle('Heuristic Value Heatmaps - All Nodes', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

```

```

=== Heuristic Verification (Assignment Example: node (3,3)) ===
Manhattan |3-6|+|3-7| = 7
Box rows 3-6, cols 3-7 K=20 cells, sum=44
Avg window cost = 44/20 = 2.20
h2(3,3) = 7 x 2.20 = 15.40 (expected 15.40)
h1(3,3) = sqrt(48+16) = 5.0000

```



In [4]: # — Cell 4 : GBFS Algorithm —————

```

def gbfs(grid, start, goal, heuristic_fn):
    """
    Greedy Best-First Search.
    Priority queue entry: (h_value, dir_priority, counter, node, parent)
    Tie-break: lower dir_priority (North=0, East=1, South=2, West=3)
    Returns: (path | None, iteration_log, explored_set, h_values)
    """
    open_heap = []
    closed_set = set()
    came_from = {start: None}
    h_values = {}
    log = []
    counter = 0
    iteration = 0

    h0 = heuristic_fn(start)
    h_values[start] = h0
    heapq.heappush(open_heap, (h0, 0, counter, start, None))

    while open_heap:
        h_cur, _dp, _cnt, current, parent = heapq.heappop(open_heap)

        if current in closed_set:
            continue

        came_from[current] = parent
        closed_set.add(current)
        iteration += 1

        frontier_snap = {item[3] for item in open_heap}
        entry = {
            'iter' : iteration,
            'current' : current,
            'h_value' : h_cur,
            'frontier' : set(frontier_snap),
            'explored' : set(closed_set),
        }

        if current == goal:
            entry['status'] = 'GOAL REACHED'
            log.append(entry)
            path = []
            node = goal
            while node is not None:
                path.append(node)
                node = came_from[node]
            path.reverse()
            return path, log, closed_set, h_values

    r, c = current
    nbrs = []
    for dr, dc, dname in DIRECTIONS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS:
            cell = grid[nr][nc]
            if cell != 'N' and (nr, nc) not in closed_set:
                h_n = heuristic_fn((nr, nc))

```

```

        h_values[(nr, nc)] = h_n
        counter += 1
        heapq.heappush(open_heap,
                        (h_n, DIR_PRI[dname], counter, (nr, nc), curr
                        nbrs.append(((nr, nc), h_n, dname))

    entry['neighbors'] = nbrs
    if open_heap:
        nxt = min(open_heap, key=lambda x: (x[0], x[1]))
        entry['next_node'] = nxt[3]
        entry['next_h'] = nxt[0]
    entry['status'] = 'EXPLORING'
    log.append(entry)

    entry = {'iter': iteration, 'current': None, 'h_value': 0,
            'frontier': set(), 'explored': set(closed_set), 'status': 'NO PATH'}
    log.append(entry)
    return None, log, closed_set, h_values

def path_cost(path, grid):
    return sum(CELL_COST.get(grid[r][c], 1) for r, c in path) if path else 0

def count_hazards(path, grid):
    w = sum(1 for r, c in path if grid[r][c] == 'W')
    n = sum(1 for r, c in path if grid[r][c] == 'N')
    return w, n

print('GBFS algorithm defined. ✓')
print('Helper functions defined: path_cost(), count_hazards() ✓')

```

GBFS algorithm defined. ✓

Helper functions defined: path_cost(), count_hazards() ✓

```

In [5]: # — Cell 5 : Run GBFS with h1 (Euclidean Distance) —————
print('=' * 65)
print('      GBFS — Heuristic h1 (Euclidean Distance)')
print('=' * 65)

tracemalloc.start()
t0 = time.perf_counter()
path_h1, log_h1, explored_h1, hvals_h1 = gbfs(RAW_GRID, START, GOAL, h1)
t1 = time.perf_counter()
_cur, peak_h1 = tracemalloc.get_traced_memory()
tracemalloc.stop()
rt_h1 = (t1 - t0) * 1000

# — Iteration table —
print(f"\n{'Iter':>4} | {'Node':>8} | {'h(n)':>8} | {'Next Node':>10} | {'h(next\n      f"{'Frontier':>8} | {'Explored':>8} | Status")
print('-' * 88)
for e in log_h1:
    ns = str(e.get('next_node', '-'))
    nh = f"{e.get('next_h', 0):.3f}" if 'next_h' in e else '-'
    print(f"{'e[\'iter\']':>4} | {str(e['current']):>8} | {e['h_value']:>8.3f} | "\n      f"{'ns':>10} | {'nh':>8} | {'len(e['frontier']):>8} | "\n      f"{'len(e['explored']):>8} | {e['status']}"")

```

```
# — Results —
w1, n1 = count_hazards(path_h1, RAW_GRID)
cost_h1 = path_cost(path_h1, RAW_GRID)
print(f'\n Path : {" → ".join(str(n) for n in path_h1)}')
print(f' Length: {len(path_h1)} nodes | Total Cost: {cost_h1}')
print(f' Nodes Explored : {len(explored_h1)}')
print(f' Weather Hazards (W) : {w1} (penalty +{w1*4})')
print(f' No-Fly Zones (N) : {n1}')
print(f' Runtime : {rt_h1:.4f} ms')
print(f' Peak Memory : {peak_h1/1024:.2f} KB')

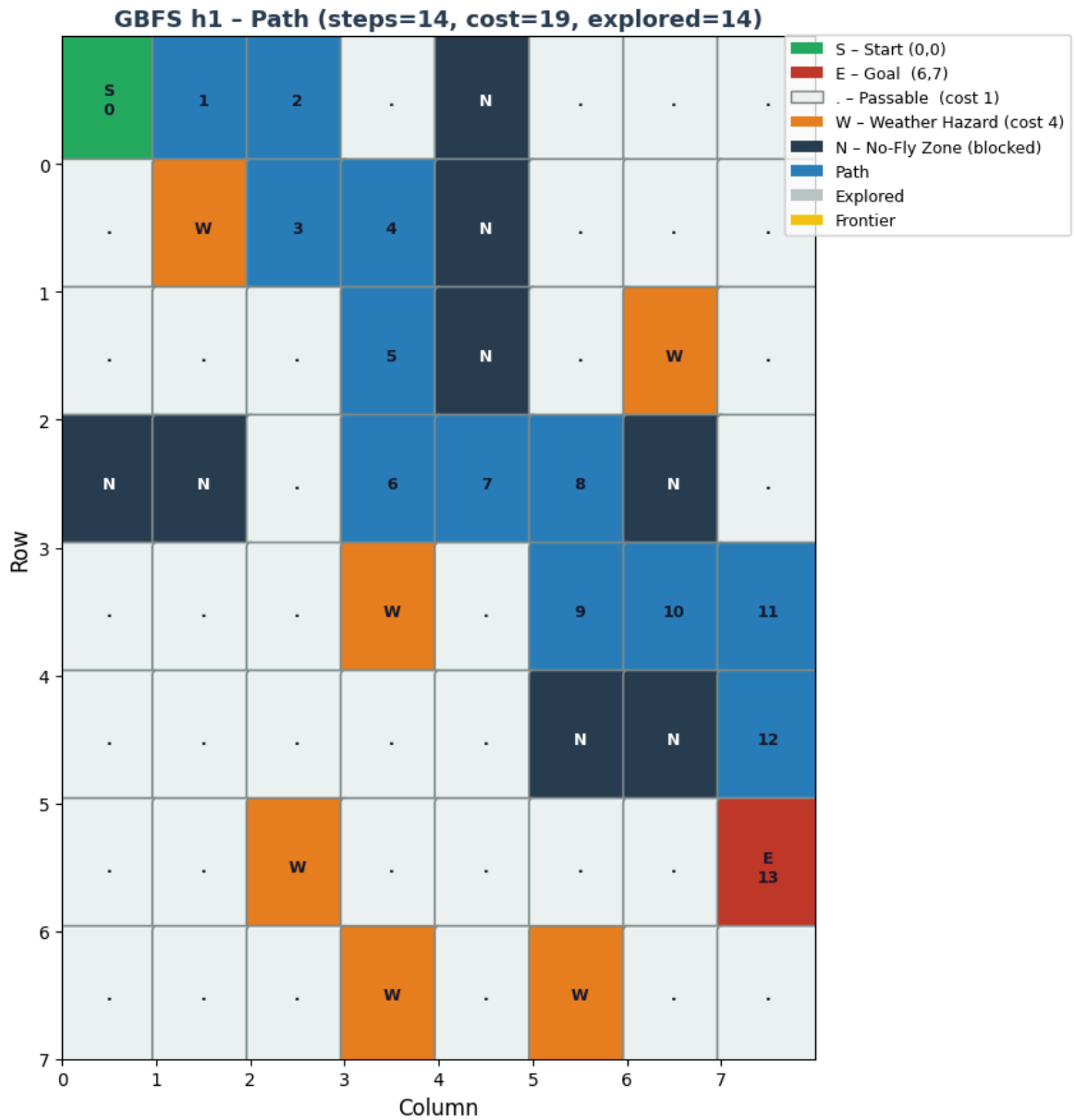
visualize_grid(RAW_GRID, path=path_h1, explored=explored_h1,
               title=f'GBFS h1 - Path (steps={len(path_h1)}, cost={cost_h1}, exp
```

```
=====
GBFS – Heuristic h1 (Euclidean Distance)
=====
```

Iter	Node	h(n)	Next Node	h(next)	Frontier	Explored	Status
1	(0, 0)	9.220	(0, 1)	8.485	0	1	EXPLOR
2	(0, 1)	8.485	(0, 2)	7.810	1	2	EXPLOR
3	(0, 2)	7.810	(1, 2)	7.071	2	3	EXPLOR
4	(1, 2)	7.071	(1, 3)	6.403	3	4	EXPLOR
5	(1, 3)	6.403	(2, 3)	5.657	4	5	EXPLOR
6	(2, 3)	5.657	(3, 3)	5.000	4	6	EXPLOR
7	(3, 3)	5.000	(3, 4)	4.243	4	7	EXPLOR
8	(3, 4)	4.243	(3, 5)	3.606	6	8	EXPLOR
9	(3, 5)	3.606	(4, 5)	2.828	7	9	EXPLOR
10	(4, 5)	2.828	(4, 6)	2.236	8	10	EXPLOR
11	(4, 6)	2.236	(4, 7)	2.000	8	11	EXPLOR
12	(4, 7)	2.000	(5, 7)	1.000	8	12	EXPLOR
13	(5, 7)	1.000	(6, 7)	0.000	9	13	EXPLOR
14	(6, 7)	0.000	–	–	9	14	GOAL R

EACHED

```
Path : (0, 0) → (0, 1) → (0, 2) → (1, 2) → (1, 3) → (2, 3) → (3, 3) → (3, 4) →
(3, 5) → (4, 5) → (4, 6) → (4, 7) → (5, 7) → (6, 7)
Length: 14 nodes | Total Cost: 19
Nodes Explored : 14
Weather Hazards (W) : 1 (penalty +4)
No-Fly Zones (N) : 0
Runtime : 0.6872 ms
Peak Memory : 40.55 KB
```

```
In [6]: # — Cell 6 : Run GBFS with h2 (Bounding-Box Risk-Weighted) —————
print('=' * 65)
print('  GBFS - Heuristic h2 (Bounding-Box Risk-Weighted)')
print('=' * 65)

tracemalloc.start()
t0 = time.perf_counter()
path_h2, log_h2, explored_h2, hvals_h2 = gbfs(RAW_GRID, START, GOAL, h2)
t1 = time.perf_counter()
_cur, peak_h2 = tracemalloc.get_traced_memory()
tracemalloc.stop()
rt_h2 = (t1 - t0) * 1000

print(f"\n{'Iter':>4} | {'Node':>8} | {'h(n)':>8} | {'Next Node':>10} | {'h(next\n      f"{'Frontier':>8} | {'Explored':>8} | Status")
print('-' * 88)
for e in log_h2:
    ns = str(e.get('next_node', '-'))
    nh = f"{e.get('next_h', 0):.3f}" if 'next_h' in e else '-'
    print(f"{'e[\'iter\']':>4} | {str(e['current']):>8} | {e['h_value']:>8.3f} | "\n          f"{'ns':>10} | {'nh':>8} | {'len(e['frontier']):>8} | "\n          f"{'len(e['explored']):>8} | {'e['status']}"")
```

```

w2, n2 = count_hazards(path_h2, RAW_GRID)
cost_h2 = path_cost(path_h2, RAW_GRID)
print(f'\n Path : {" → ".join(str(n) for n in path_h2)}')
print(f' Length: {len(path_h2)} nodes | Total Cost: {cost_h2}')
print(f' Nodes Explored : {len(explored_h2)}')
print(f' Weather Hazards (W) : {w2} (penalty +{w2*4})')
print(f' No-Fly Zones (N) : {n2}')
print(f' Runtime : {rt_h2:.4f} ms')
print(f' Peak Memory : {peak_h2/1024:.2f} KB')

visualize_grid(RAW_GRID, path=path_h2, explored=explored_h2,
               title=f'GBFS h2 - Path (steps={len(path_h2)}, cost={cost_h2}, exp

```

```

=====
GBFS – Heuristic h2 (Bounding-Box Risk-Weighted)
=====

```

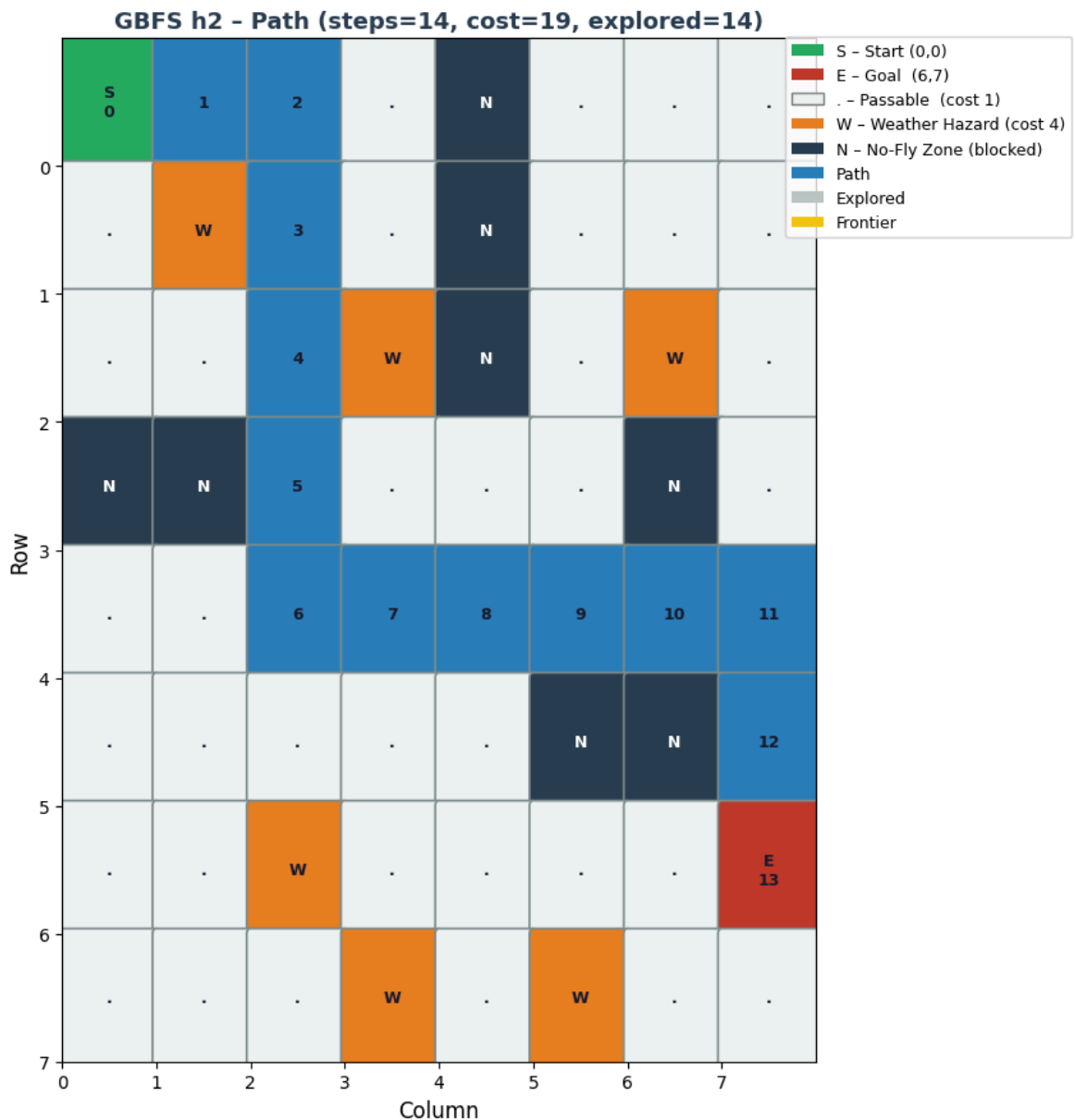
Iter	Node	h(n)	Next Node	h(next)	Frontier	Explored	Status
1	(0, 0)	29.482	(0, 1)	27.673	0	1	EXPLOR
2	(0, 1)	27.673	(0, 2)	25.143	1	2	EXPLOR
3	(0, 2)	25.143	(1, 2)	23.056	2	3	EXPLOR
4	(1, 2)	23.056	(2, 2)	21.000	3	4	EXPLOR
5	(2, 2)	21.000	(3, 2)	17.000	4	5	EXPLOR
6	(3, 2)	17.000	(4, 2)	14.778	6	6	EXPLOR
7	(4, 2)	14.778	(4, 3)	12.800	7	7	EXPLOR
8	(4, 3)	12.800	(4, 4)	10.833	9	8	EXPLOR
9	(4, 4)	10.833	(4, 5)	10.222	10	9	EXPLOR
10	(4, 5)	10.222	(4, 6)	6.500	12	10	EXPLOR
11	(4, 6)	6.500	(4, 7)	2.000	13	11	EXPLOR
12	(4, 7)	2.000	(5, 7)	1.000	13	12	EXPLOR
13	(5, 7)	1.000	(6, 7)	0.000	14	13	EXPLOR
14	(6, 7)	0.000	–	–	14	14	GOAL R

EACHED

```

Path : (0, 0) → (0, 1) → (0, 2) → (1, 2) → (2, 2) → (3, 2) → (4, 2) → (4, 3) →
(4, 4) → (4, 5) → (4, 6) → (4, 7) → (5, 7) → (6, 7)
Length: 14 nodes | Total Cost: 19
Nodes Explored : 14
Weather Hazards (W) : 1 (penalty +4)
No-Fly Zones (N) : 0
Runtime : 1.0764 ms
Peak Memory : 35.28 KB

```



```
In [7]: # — Cell 7 : A* Implementation & Comparison with GBFS —————

def astar(grid, start, goal, heuristic_fn):
    """
    A* Search using f = g + h.
    Returns: (path | None, nodes_expanded, runtime_ms, peak_mem_kb)
    """
    open_heap = []
    closed_set = set()
    g_cost = {start: 0}
    came_from = {start: None}
    counter = 0

    heapq.heappush(open_heap, (heuristic_fn(start), 0, counter, start))

    tracemalloc.start()
    t0 = time.perf_counter()

    while open_heap:
        f, _dp, _cnt, current = heapq.heappop(open_heap)
        if current in closed_set:
            continue
        closed_set.add(current)
```

```

        if current == goal:
            t1 = time.perf_counter()
            _, peak = tracemalloc.get_traced_memory()
            tracemalloc.stop()
            path = []
            node = goal
            while node is not None:
                path.append(node); node = came_from[node]
            path.reverse()
            return path, len(closed_set), (t1 - t0) * 1000, peak / 1024

    r, c = current
    for dr, dc, dname in DIRECTIONS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < ROWS and 0 <= nc < COLS:
            cell = grid[nr][nc]
            if cell != 'N' and (nr, nc) not in closed_set:
                new_g = g_cost[current] + CELL_COST.get(cell, 1)
                if (nr, nc) not in g_cost or new_g < g_cost[(nr, nc)]:
                    g_cost[(nr, nc)] = new_g
                    came_from[(nr, nc)] = current
                    counter += 1
                    heapq.heappush(open_heap,
                                   (new_g + heuristic_fn((nr, nc)),
                                    DIR_PRI[dname], counter, (nr, nc)))

    tracemalloc.stop()
    return None, len(closed_set), 0, 0

print('=' * 65)
print('          A* - Heuristic h1 (Euclidean Distance)')
print('=' * 65)
path_as, nodes_as, rt_as, mem_as = astar(RAW_GRID, START, GOAL, h1)
wa, _ = count_hazards(path_as, RAW_GRID)
cost_as = path_cost(path_as, RAW_GRID)
print(f' Path : {" → ".join(str(n) for n in path_as)}')
print(f' Length: {len(path_as)} | Cost: {cost_as} | Nodes Expanded: {nodes_as}')
print(f' Weather Hazards: {wa} | Runtime: {rt_as:.4f} ms | Peak Mem: {mem_as}')

visualize_grid(RAW_GRID, path=path_as,
               title=f'A* h1 - Optimal Path (steps={len(path_as)}, cost={cost_as})

# — Comparison Table —
print('\n' + '=' * 70)
print('          Algorithm Comparison Table')
print('=' * 70)
df = pd.DataFrame({
    'Algorithm'      : ['GBFS h1', 'GBFS h2', 'A* h1'],
    'Nodes Expanded' : [len(explored_h1), len(explored_h2), nodes_as],
    'Path Length'    : [len(path_h1), len(path_h2), len(path_as)],
    'Path Cost'      : [cost_h1, cost_h2, cost_as],
    'W-Hazards'      : [w1, w2, wa],
    'Runtime (ms)'    : [round(rt_h1, 4), round(rt_h2, 4), round(rt_as, 4)],
    'Peak Mem (KB)'   : [round(peak_h1/1024, 2), round(peak_h2/1024, 2), round(mem_as/1024, 2)],
    'Complete'       : ['No', 'No', 'Yes'],
    'Optimal'        : ['No', 'No', 'Yes'],
})
print(df.to_string(index=False))

```

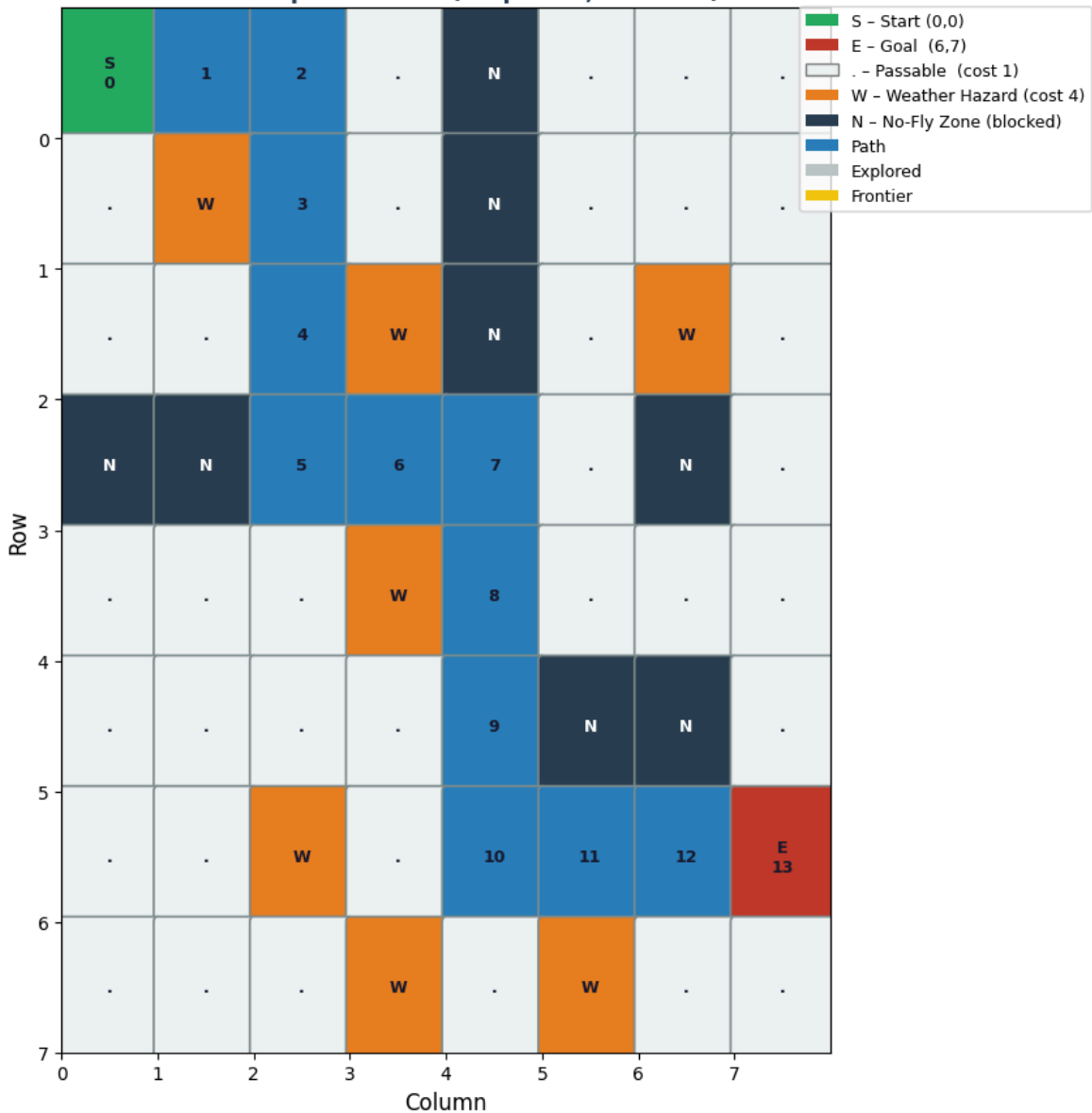
=====

A* – Heuristic h1 (Euclidean Distance)

=====

Path : (0, 0) → (0, 1) → (0, 2) → (1, 2) → (2, 2) → (3, 2) → (3, 3) → (3, 4) →
 (4, 4) → (5, 4) → (6, 4) → (6, 5) → (6, 6) → (6, 7)
 Length: 14 | Cost: 16 | Nodes Expanded: 32
 Weather Hazards: 0 | Runtime: 0.4003 ms | Peak Mem: 8.20 KB

A* h1 - Optimal Path (steps=14, cost=16)



=====

Algorithm Comparison Table

=====

Algorithm	Nodes Expanded	Path Length	Path Cost	W-Hazards	Runtime (ms)	Peak Mem (KB)
GBFS h1	Complete	14	19	1	0.6872	40.55
GBFS h2	No	14	19	1	1.0764	35.28
A* h1	No	32	16	0	0.4003	8.20
	Yes	Yes				

```
In [8]: # — Cell 8 : Complexity Analysis —————
print('=' * 65)
print('Complexity Analysis')
print('=' * 65)
```

```

metrics = pd.DataFrame({
    'Metric'      : ['Nodes Expanded', 'Runtime (ms)', 'Memory (OPEN+CLOSED)',
                    'Total Path Cost', 'Path Length', 'h(start)'],
    'GBFS h1'     : [len(explored_h1), f'{rt_h1:.4f}',
                    f'{len(explored_h1)} nodes', cost_h1, len(path_h1),
                    f'{h1(START):.3f}'],
    'GBFS h2'     : [len(explored_h2), f'{rt_h2:.4f}',
                    f'{len(explored_h2)} nodes', cost_h2, len(path_h2),
                    f'{h2(START):.3f}'],
    'A* h1'       : [nodes_as, f'{rt_as:.4f}',
                    f'{nodes_as} nodes', cost_as, len(path_as),
                    f'{h1(START):.3f}'],
})
print(metrics.to_string(index=False))
print()
print('  Time Complexity (Big-O):')
print('    GBFS :  $O(b^m)$  –  $b$  = max branching factor ( $\leq 4$ ),  $m$  = max depth ( $\leq 64$ )')
print('    A*   :  $O(b^d)$  –  $d$  = optimal solution depth (admissible heuristic)')
print('    Both use a binary-heap priority queue  $\rightarrow O(\log n)$  per push/pop')
print()
print('  Space Complexity:')
print('    GBFS :  $O(b^m)$  – keeps entire frontier in memory')
print('    A*   :  $O(b^d)$  – keeps frontier + explored set')

```

```

=====
                        Complexity Analysis
=====

```

	Metric	GBFS h1	GBFS h2	A* h1
	Nodes Expanded	14	14	32
	Runtime (ms)	0.6872	1.0764	0.4003
Memory (OPEN+CLOSED)	14 nodes	14 nodes	32 nodes	
	Total Path Cost	19	19	16
	Path Length	14	14	14
	h(start)	9.220	29.482	9.220

Time Complexity (Big-O):

GBFS : $O(b^m)$ – b = max branching factor (≤ 4), m = max depth (≤ 64)

A* : $O(b^d)$ – d = optimal solution depth (admissible heuristic)

Both use a binary-heap priority queue $\rightarrow O(\log n)$ per push/pop

Space Complexity:

GBFS : $O(b^m)$ – keeps entire frontier in memory

A* : $O(b^d)$ – keeps frontier + explored set

```

In [ ]: # — Cell 9 : Trap Analysis —————
print('=' * 65)
print('          Trap Analysis - Local Optima in GBFS')
print('=' * 65)
print('  A trap = node whose best available neighbour has a HIGHER')
print('  (or equal) heuristic than the current node (greedy regress).')
print('  GBFS handles this by falling back to the global open list.\n')

def find_traps(log):
    traps = []
    for e in log:
        if e['status'] != 'EXPLORING':
            continue
        nbrs = e.get('neighbors', [])
        if not nbrs:
            traps.append({

```

```

        'Iter': e['iter'], 'Node': str(e['current']),
        'h(cur)': f"{e['h_value']:.3f}",
        'Best nbr h': 'N/A', 'Type': 'Dead-End',
        'Resolution': 'Backtrack via open list',
    })
else:
    best_h = min(h for _, h, _ in nbrs)
    if best_h >= e['h_value']:
        traps.append({
            'Iter': e['iter'], 'Node': str(e['current']),
            'h(cur)': f"{e['h_value']:.3f}",
            'Best nbr h': f'{best_h:.3f}',
            'Type': 'Local Minimum',
            'Resolution': 'Continue via open list',
        })
return traps

traps_h1 = find_traps(log_h1)
traps_h2 = find_traps(log_h2)

for hname, traps in [('h1', traps_h1), ('h2', traps_h2)]:
    print(f' Traps for GBFS {hname}: {len(traps)} detected')
    if traps:
        df_t = pd.DataFrame(traps)
        print(df_t.to_string(index=False))
    print()

print(' Key insight: GBFS is not complete – it CAN get stuck in')
print(' infinite loops on graphs without visited-state checking.')
print(' Our implementation uses a closed set to prevent re-visits.')

```

In [9]: # — Cell 10 : Heuristic Analysis Visualisations —————

```

fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# Plot 1 - Heuristic along path
ax = axes[0, 0]
ax.plot([h1(n) for n in path_h1], 'b-o', ms=5, label='GBFS h1')
ax.plot([h2(n) for n in path_h2], 'r-s', ms=5, label='GBFS h2')
ax.plot([h1(n) for n in path_as], 'g-^', ms=5, label='A* h1')
ax.set_title('Heuristic Values Along Path', fontweight='bold')
ax.set_xlabel('Step Index'); ax.set_ylabel('Heuristic h(n)')
ax.legend(); ax.grid(True, alpha=0.3)

# Plot 2 - Nodes expanded bar chart
ax = axes[0, 1]
algos = ['GBFS h1', 'GBFS h2', 'A* h1']
n_exp = [len(explored_h1), len(explored_h2), nodes_as]
colors = ['#3498db', '#e74c3c', '#2ecc71']
bars = ax.bar(algos, n_exp, color=colors, edgecolor='black', alpha=0.85)
for b, v in zip(bars, n_exp):
    ax.text(b.get_x() + b.get_width()/2, b.get_height() + 0.15,
            str(v), ha='center', fontweight='bold')
ax.set_title('Nodes Expanded', fontweight='bold')
ax.set_ylabel('Count'); ax.grid(True, alpha=0.3, axis='y')

# Plot 3 - Path cost bar chart
ax = axes[1, 0]
costs = [cost_h1, cost_h2, cost_as]
bars = ax.bar(algos, costs, color=colors, edgecolor='black', alpha=0.85)

```

```

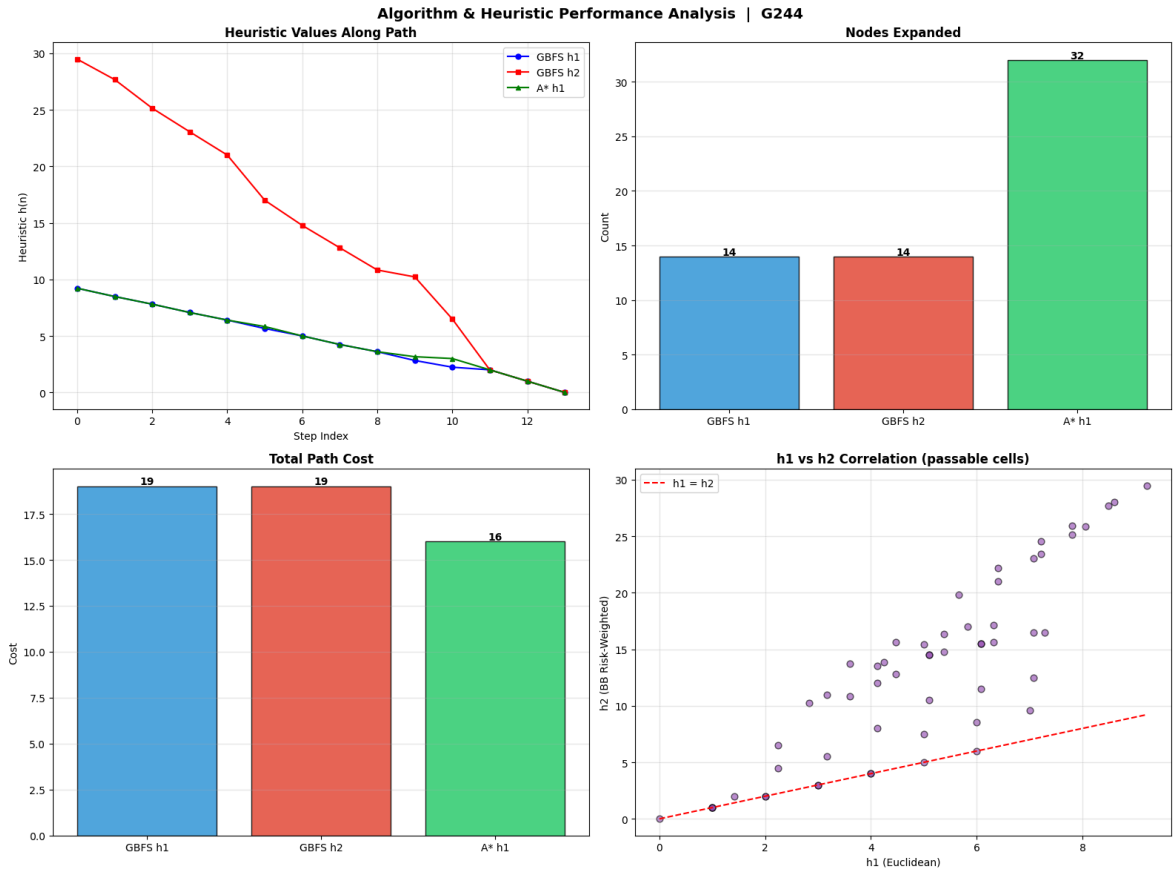
for b, v in zip(bars, costs):
    ax.text(b.get_x() + b.get_width()/2, b.get_height() + 0.1,
            str(v), ha='center', fontweight='bold')
ax.set_title('Total Path Cost', fontweight='bold')
ax.set_ylabel('Cost'); ax.grid(True, alpha=0.3, axis='y')

# Plot 4 - Heuristic value distribution (scatter)
ax = axes[1, 1]
all_h1 = [h1((r, c)) for r in range(ROWS) for c in range(COLS)
           if RAW_GRID[r][c] != 'N']
all_h2 = [h2((r, c)) for r in range(ROWS) for c in range(COLS)
           if RAW_GRID[r][c] != 'N']
ax.scatter(all_h1, all_h2, alpha=0.6, color='#8e44ad', edgecolors='black', s=40)
ax.plot([0, max(all_h1)], [0, max(all_h1)], 'r--', label='h1 = h2')
ax.set_title('h1 vs h2 Correlation (passable cells)', fontweight='bold')
ax.set_xlabel('h1 (Euclidean)'); ax.set_ylabel('h2 (BB Risk-Weighted)')
ax.legend(); ax.grid(True, alpha=0.3)

plt.suptitle('Algorithm & Heuristic Performance Analysis | G244',
             fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('=== Heuristic Function Analysis ===')
print(f' h1: avg = {sum(all_h1)/len(all_h1):.2f} min = {min(all_h1):.2f} max')
print(f' h2: avg = {sum(all_h2)/len(all_h2):.2f} min = {min(all_h2):.2f} max')
print()
if cost_h2 < cost_h1:
    print(f' h2 produced LOWER path cost ({cost_h2} vs {cost_h1}).')
    print(' h2 is risk-aware: it penalises nodes near hazard clusters,')
    print(' steering the agent towards safer corridors.')
elif cost_h1 < cost_h2:
    print(f' h1 produced LOWER path cost ({cost_h1} vs {cost_h2}).')
    print(' Euclidean distance provides a tight admissible lower bound')
    print(' and avoids the over-estimation risk of h2 near hazards.')
else:
    print(f' Both heuristics produced equal path cost = {cost_h1}.')
print()
print(' Summary:')
print(' h1 - Fast, admissible, geometry-only; may cross hazards')
print(' h2 - Risk-aware, future-looking; costlier to compute')
print(' A* - Optimal & complete; explores more nodes than GBFS')

```

=== Heuristic Function Analysis ===

h1: avg = 4.95 min = 0.00 max = 9.22

h2: avg = 12.82 min = 0.00 max = 29.48

Both heuristics produced equal path cost = 19.

Summary:

h1 - Fast, admissible, geometry-only; may cross hazards

h2 - Risk-aware, future-looking; costlier to compute

A* - Optimal & complete; explores more nodes than GBFS

```
In [10]: # — Cell 11 : Write Output File —————

def write_output(filepath):
    with open(filepath, 'w') as f:
        f.write('=' * 65 + '\n')
        f.write('  Output Report - Defence Drone GBFS | Group G244\n')
        f.write('=' * 65 + '\n\n')

    for algo, path, llog, expl, rt, pm, wz in [
        ('GBFS h1 (Euclidean)', path_h1, log_h1, explored_h1, rt_h1, peak_h1,
        ('GBFS h2 (BB Risk-Weighted)', path_h2, log_h2, explored_h2, rt_h2,
        ]:
        f.write(f'Algorithm      : {algo}\n')
        f.write(f'Path          : {" -> ".join(str(n) for n in path)}\n')
        f.write(f'Path Length   : {len(path)}\n')
        f.write(f'Total Path Cost : {path_cost(path, RAW_GRID)}\n')
        f.write(f'Nodes Explored : {len(expl)}\n')
        f.write(f'Weather Hazards : {wz} (penalty +{wz*4})\n')
        f.write(f'Runtime       : {rt:.4f} ms\n')
        f.write(f'Peak Memory    : {pm/1024:.2f} KB\n')
        f.write('\nIteration Log:\n')
        f.write(f'{"Iter":>4} {"Node":>8} {"h(n)":>8} Status\n')
        f.write('-' * 38 + '\n')
```

```

    for e in llog:
        f.write(f"{e['iter']:>4} {str(e['current']):>8} "
               f"{e['h_value']:>8.3f} {e['status']}\n")
    f.write('\n' + '-' * 65 + '\n\n')

    f.write(f'Algorithm      : A* (h1)\n')
    f.write(f'Path            : {" -> ".join(str(n) for n in path_as)}\n')
    f.write(f'Path Length       : {len(path_as)}\n')
    f.write(f'Total Path Cost  : {cost_as}\n')
    f.write(f'Nodes Expanded    : {nodes_as}\n')
    f.write(f'Runtime           : {rt_as:.4f} ms\n')
    f.write(f'Peak Memory       : {mem_as:.2f} KB\n')

write_output(OUTPUT_FILE)
print(f'Output written → {OUTPUT_FILE}')
print()
print('-' * 55)
print('  FINAL SUMMARY')
print('-' * 55)
print(f'  GBFS h1 : {len(path_h1)} steps, cost={cost_h1}, explored={len(explored)}')
print(f'  GBFS h2 : {len(path_h2)} steps, cost={cost_h2}, explored={len(explored)}')
print(f'  A* h1   : {len(path_as)} steps, cost={cost_as}, explored={nodes_as}')
print('-' * 55)

```

Output written → c:\Users\Priya_Das\CascadeProjects\G244_A1_PS4\outputPS4.txt

FINAL SUMMARY

GBFS h1 : 14 steps, cost=19, explored=14
 GBFS h2 : 14 steps, cost=19, explored=14
 A* h1 : 14 steps, cost=16, explored=32

Conclusion

Property	GBFS h1	GBFS h2	A* h1
Complete	X	X	✓
Optimal	X	X	✓
Uses future risk	X	✓	X
Speed	Fast	Fast	Slower

Key findings:

- **GBFS** is fast but greedy — neither complete nor optimal. It relies entirely on the heuristic to steer toward the goal.
- **h1 (Euclidean)** is a classic admissible lower bound; it ignores terrain hazards.
- **h2 (Bounding-Box Risk-Weighted)** is future-aware; it accounts for the average hazard density between the agent and the goal, potentially routing around dangerous regions.
- **A*** with h1 guarantees the *optimal* (least-cost) path, at the expense of exploring more nodes.
- For real-time drone operations where speed matters: **GBFS h2** is preferable.

- For mission-critical scenarios requiring guaranteed optimality: **A* h1** is the right choice.

Group G244 | BITS Pilani WILPD | MTech AI & ML | S2 2025-2026